Numerical and Architectural Challenges in Porting Granite-4.0-h Small/Tiny Models from HuggingFace to Tenstorrent Wormhole (Galaxy 32) Architecture

MSc AI & Data Engineering Research Project

Anonymous

Abstract

Running large language models efficiently today almost always means using NVIDIA GPUs. This project asks whether a fundamentally different type of AI accelerator – Tenstorrent Wormhole – can match or exceed GPU performance for real production models, and documents the engineering work required to find out.

I port IBM's Granite-4.0-h (tiny and small variants) from the standard HuggingFace/PyTorch stack to Tenstorrent's Wormhole hardware, specifically a Galaxy 32 multi-chip system. The model combines recurrent and attention layers with sparse expert routing, making it a demanding and representative inference target.

The porting work surfaces three main challenges: maintaining numeric correctness when compressing model weights to fit within device memory, restructuring the model to keep data on-chip across all layers without round-tripping through the host, and working around operations that the hardware's software library does not yet support. The dominant performance bottleneck is that the host CPU must send a separate command to the device for every operation, and this communication cost – not the arithmetic – limits throughput. I address this with three proposed optimisations. First, I record the full forward pass for one token once and replay it with a single command. Second, I fuse the small operations that update the model's running memory state. Third, I fuse the repeated scale-and-add step used in every layer.

With trace enabled, the tiny model (4 Wormhole chips) achieves 10.65–10.69 tok/s decode throughput – exceeding an NVIDIA A100 GPU (8.5–9.0 tok/s) by ≈18% and ≈4× faster than CPU. The small model (8 chips, trace enabled) achieves 5.84–5.89 tok/s, which is below the A100 (7.1–8.2 tok/s) by ≈20% due to DRAM bandwidth constraints at larger model scale, but ≈9× faster than CPU. These results show that non-GPU hardware can exceed A100 throughput for smaller models; the challenges and solutions are described in detail for future practitioners.

1 Introduction

Large language models (LLMs) have become a fundamental component of modern AI systems, with most implementations heavily optimized for GPU-based platforms such as NVIDIA A100. However, the increasing demand for efficient inference and cost-effective deployment has led to the emergence of alternative AI accelerators, including the Tenstorrent Wormhole architecture.

This project explores porting Granite-4.0-h small and tiny models – IBM's hybrid language models combining recurrent and attention layers with sparse expert routing – from the standard HuggingFace/PyTorch stack to Tenstorrent Wormhole on Galaxy 32. On a GPU, hundreds of operations in a forward pass run efficiently with mature GPU runtimes. On Wormhole, each operation must be explicitly sent from the host CPU to the device one at a time. Each send takes a fixed time regardless of how much computation the operation actually performs. During token generation – where the model processes just one token at a time – this per-operation cost accumulates and becomes the main performance limit. Section 2.1 describes the hardware in detail.

The key research questions addressed in this work are:

- What numerical and architectural challenges arise when porting a hybrid recurrent-attention model with sparse expert routing from HuggingFace to Wormhole?
- Why does the cost of sending each operation from the host to the device become the main bottleneck during token generation, and which optimisations reduce this overhead?
- When model weights are split across several chips, what additional communication and memory constraints must the implementation satisfy to keep generation fast and correct?

The contributions of this project are:

- Complete working implementations of Granite-4.0-h-tiny (4 Wormhole chips) and Granite-4.0-h-small (8 chips), with token generation throughput benchmarked against NVIDIA A100 and CPU baselines.
- A systematic description of the numerical, architectural, and toolchain challenges encountered during the port (Section 4).
- A custom on-chip kernel for the recurrent state-update that reduces memory traffic and command count per token.
- A working implementation of whole-model trace recording and replay, where the full per-token forward pass is recorded once and replayed with a single command per token.
- Empirical token generation throughput measurements and per-component timing analysis.

2 Background

2.1 Tenstorrent Wormhole Platform

2.1.1 Galaxy 32: the multi-chip system. The system used in this work is Tenstorrent's Galaxy 32, a machine containing 32 Wormhole chips arranged in a 4×8 mesh [Tenstorrent 2023a]. The chips are connected by built-in inter-chip links, so data can move between chips without going through the host CPU. This is important for splitting a large model across many chips and combining results on-device during inference.

The system software uses these links to move data between chips, for example to gather partial results from many chips and combine them into one final output. In this work, communication is routed along one row or one column of the mesh at a time. This is required

for hardware trace replay (Section 5.3): the full 2D routing mode is not trace-safe on this hardware stack, whereas single-axis (1D) routing is; Section 5.3 explains the constraint.

The host machine connects to the Galaxy 32 over PCIe. From the host's point of view, the 32-chip system looks like a single device. The host sends commands over the PCIe link, and the chips work together through the built-in chip network to carry them out.

2.1.2 Single Wormhole chip architecture. Each of the 32 chips is a Wormhole chip – Tenstorrent's second-generation AI accelerator [Tenstorrent 2023b]. A single chip contains a 2D grid of compute cores arranged in an 8×10 grid. Of these grid positions, 64 are programmable worker cores (called Tensix cores) that execute AI kernels; the remaining positions are occupied by dedicated hardware for other tasks: 6 DRAM controllers (circuits that manage data transfers between the chip and its off-chip memory), 16 Ethernet cores (small processors that handle chip-to-chip network traffic), and an ARC management processor (a low-power core that handles boot, power, and device initialisation).

2.1.3 Memory hierarchy. **L1 (on-chip SRAM).** Each Tensix core has 1.43 MiB of on-chip SRAM called L1 [Tenstorrent 2023b]. The 64 compute cores provide 91.5 MiB total worker L1 per chip. Unlike a CPU's hardware cache which automatically copies data in and out, L1 here is software-managed: the programmer must explicitly load data into L1 before using it and write results back to DRAM afterwards. Programs that run on a Tensix core are called *kernels*. When one computation feeds directly into the next, the result would normally be written to DRAM only to be read back immediately – wasting bandwidth. Tenstorrent's Metal programming framework avoids this by letting the programmer designate named regions of L1 as *circular buffers*: the first kernel writes its result into a labelled L1 slot, and the next kernel reads from that same slot, never touching DRAM for that intermediate value.

DRAM (off-chip). Each chip has 12 GB of off-chip DRAM for storing model weights and activations [Tenstorrent 2023b]. TTNN organises tensor data (e.g. a weight matrix) into fixed 32×32-element blocks called *tiles* – the smallest unit a Tensix core can request from DRAM. This work uses interleaved layout for all weight matrices and activations: tiles are scattered across DRAM so many cores can fetch different parts of the same matrix in parallel.

Table 1 summarises the memory tiers on a single chip.

Table 1: Memory tiers on a single Wormhole chip [Tenstorrent 2023b].

Level	Size
L1 (per core)	1.43 MiB
L1 (chip total)	91.5 MiB
DRAM (per chip)	12 GB

2.1.4 TTNN execution model. The host CPU triggers each operation on a Wormhole chip by writing a command over the PCIe link – one command per operation (e.g. a matrix multiply, an element-wise multiply). Each command incurs a fixed cost to send it over PCIe, regardless of how much arithmetic the operation actually performs. When generating one token at a time, the model issues many such commands per token, and the cumulative cost of sending them all dominates the time to generate a single token, making the PCIe link the bottleneck rather than the compute hardware itself.

Hardware trace (replay). TTNN lets you record a full sequence of operations once, then replay the entire sequence later with just one command, eliminating the per-command PCIe cost. While recording, operations are stored but not executed on the device; Section 4 discusses why this matters for correctness. The implementation is in Section 5.3.

2.2 Granite-4.0-h Model Variants

This work evaluates two model variants – Granite-4.0-h-tiny [IBM Research 2025d] and Granite-4.0-h-small [IBM Research 2025a]. Both have the same structure: 40 decoder layers, where each layer first mixes sequence information (using Mamba-2 for 36 of the layers, attention for the other 4), then applies a Mixture-of-Experts (MoE) feed-forward transformation. Table 2 compares their key dimensions.

Table 2: Granite-4.0-h architectural dimensions: tiny vs. small [IBM Research 2025b,e].

	Tiny	Small
Mamba-2 blocks	36	36
Attention blocks	4	4
Hidden size H	1536	4096
Number of MoE experts	64	72
Experts active per token	6	10
Total weights	13.9 GB	64.4 GB

Total weight sizes read from `metadata.total_size` in the HuggingFace checkpoint index files [IBM Research 2025c,f].

As shown in Figure 1, every decoder layer consists of two blocks stacked in sequence. The first is a *mixer block*: either a Mamba-2 block (36 of the 40 layers) or an attention block (the remaining 4). The second is an *MoE block*: a router selects a few experts from a larger pool and combines their outputs, with a shared MLP running in parallel and added to the result.

2.3 Mamba-2 Block

Mamba-2 is a *state-space model* (SSM): instead of storing every past token in a key-value cache (as attention does), it compresses the entire conversation history into a fixed-size recurrent state [Dao and Gu 2024]. Running a language model has two phases. In the *prefill* phase, the model processes the entire input prompt, updating the recurrent state for each token in sequence. In the *decode* phase, the model generates new tokens one at a time; each step takes the token produced by the previous step, updates the fixed-size state, and produces the next output token. This work focuses on the decode phase. Each decode step through a Mamba-2 block has two parts: a *Projection* that transforms the incoming hidden state into the values needed for the memory update, and a *Selective SSM* (the recurrent memory update step) that updates the stored memory with the new token and produces the output.

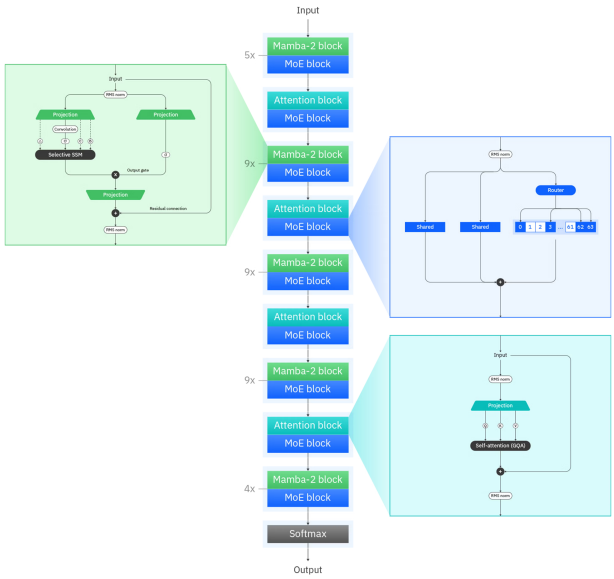

Figure 1: The Granite hybrid architecture, as implemented in Granite-4.0-H-Tiny [IBM 2025].

2.3.1 Projection. A *projection* is a learned matrix multiplication that maps the hidden state to a different set of values needed by subsequent steps. The *Input* in Figure 1 is the hidden state (the vector representing the model's current understanding at that layer). It is first normalised (RMS norm), then two parallel projections are applied.

The *left projection* produces four values:

- Δ – how much the memory should change at this step: a large Δ means the memory updates strongly; a small Δ means it mostly retains the previous state.
- σ – the content of the current token.
- C – what to look for when reading from memory: determines which parts of the stored state are extracted as output.
- B – how strongly to write the current token into memory.

σ, B, and C are concatenated and passed together through a *Convolution*: a weighted sum over a fixed window of 4 tokens – the current token and the 3 preceding ones. This window size (4) is a fixed architectural parameter of the model. To look back 3 steps, the 3 most recent values of σ, B, C are kept in a small tensor called the *convolution cache*, which is updated at every decode step. Only Δ bypasses the Convolution and goes directly to the Selective SSM.

The *right projection* produces a separate σ used as the *output gate*: after the Selective SSM produces its output, it is multiplied element-wise by this σ to selectively suppress or amplify features, then passed through a final projection back to the original hidden size.

2.3.2 Selective SSM. The Selective SSM maintains a fixed-size state $\mathbf{h}$ – a matrix that summarises all tokens seen so far. The state is split across H_m independent *heads*, similar to how attention uses multiple heads to capture different aspects of the sequence. Each head is a $D \times N$ matrix (64×128 for the tiny model): D features, each summarised into N values that encode its history across all past tokens.

Each decode step updates the state and reads an output from it in two operations:

State update (Equation 1):

$$\mathbf{h} \leftarrow \underbrace{\exp(\Delta \odot A)}_{\text{forget factor}} \odot \mathbf{h} + \underbrace{(\Delta\, \sigma) \odot B}_{\text{new token contribution}} \tag{1}$$

The forget factor (controlled by Δ and the learned matrix A) determines how much of the old state to retain: a value near 1 keeps it, near 0 discards it. The new token contribution (controlled by Δ, σ, and B) writes the current token into the state. Δ therefore acts as a gate that scales both how much to remember and how much to write.

Output read (Equation 2):

$$y = \mathbf{h} \cdot C \tag{2}$$

C selects which information to extract from the state.

2.4 MoE Block

The MoE block, shown in Figure 1, contains E experts (64 for tiny, 72 for small), each a small MLP. A *router* selects only the top-k experts per token (6 for tiny, 10 for small) and combines their outputs. The remaining experts are not executed – the model has many expert parameters but only a small fraction is activated per token.

Storing all E experts on a single chip is not feasible given DRAM constraints. Instead, I apply *expert parallelism* (EP): the expert weights are split evenly across chips, so each chip stores and executes only its local share. Since the router may select experts spread across different chips, each chip computes a *partial contribution* – it runs all its local experts, multiplies each output by its routing weight (zero for unselected experts), and sums them into a single vector. These partial vectors are then collected from all chips over the inter-chip links and summed to produce the final MoE output.

2.5 Fitting Granite-4.0-h-tiny into the Memory Hierarchy

This section checks that the tiny model fits within the 12 GB per-chip DRAM limit (Table 1). All weights are stored in bfloat16 (a 16-bit floating-point format, 2 B/element).

The dominant costs per chip are:

- **Mamba-2 projection weights** (W_{in}, W_{out}, 36 layers). Both projections from Section 2.3 are implemented as a single matrix multiply using W_{in}. Recall from Section 2.3 that H_m=48 is the number of SSM heads and D=64 is the number of features per head. W_{in} maps the hidden state (a vector of size H=1536) to all outputs at once: the output gate (size $H_m \times D = 48 \times 64 = 3072$, matching the SSM output size so it can be multiplied element-wise with it) + σ (size 3072) + B (size 128) + C (size 128) + Δ (size 48) = 6448 total. This gives a weight matrix of shape 1536×6448, costing 1536×6448×2 B ≈ 18.9 MB. W_{out} maps the SSM output (size 3072) back to the original hidden size (H=1536): shape

3072×1536, costing 3072×1536×2 B ≈ 9.0 MB. Combined, $W_{\text{in}} + W_{\text{out}} \approx 18.9 + 9.0 = 27.9$ MB per layer, ×36 layers ≈ **1 GB**.

- **Mamba-2 recurrent state** (36 layers). Three tensors must be read and updated on every decode step, so they live in DRAM and are streamed each time:
  - *SSM state* **h**: the recurrent memory from Section 2.3, shaped $H_m \times D \times N = 48 \times 64 \times 128 \rightarrow$ At 2 B/element: 48×64×128×2 B = 768 KB.
  - *Learned matrix A*: the fixed forget-factor weights from Equation 1, the same shape as **h**, 768 KB.
  - *Convolution cache*: introduced in Section 2.3, this stores the 3 most recent values of σ, B, C so the Convolution (window size 4) can cover the current token and the 3 preceding ones. Updated every decode step. Size 832 KB.

  Total per layer: 768 + 768 + 832 KB ≈ 2.32 MB. ×36 layers ≈ **83.5 MB**.
- **MoE expert weights** (64 experts split evenly across 4 chips = 16 per chip, 40 layers). Each expert's MLP has two weight matrices: a gate+up projection (maps hidden size 1536 → intermediate size 1024, so 1024×1536 elements) and a down projection (maps back 512×1536 elements), each at 2 B/element (bfloat16). Per chip per layer: 16×(1024×1536 + 512×1536)×2 B = 72 MB. ×40 layers ≈ **2.88 GB**.

The total fits comfortably within 12 GB. However, the 83.5 MB recurrent state alone nearly fills the 91.5 MiB of total L1 across all cores, and the weight matrices are far too large for L1. Consequently, all weights and recurrent state reside permanently in DRAM and must be streamed on every decode step.

3 Implementation

3.1 On-Device Execution Strategy

All 40 decoder layers run on-device end-to-end: the hidden state (a vector of size H representing the model's current understanding) is sent to the device once at the start of prefill and stays in DRAM across all layers and all decode steps. The only data that travels back to the host per step is the final logits (one score per vocabulary word): the host CPU selects the highest-scoring token, looks up its embedding, and writes that embedding back to the device as the input for the next step. Token selection runs on the host because it involves conditional logic that cannot be recorded into a fixed trace: for example, a *repetition penalty* boosts the scores of tokens not yet seen (to avoid the model repeating itself), then the highest-scoring token is selected. The transfer is small: the logits tensor has one value per vocabulary word (100,352 words [IBM Research 2025e] × 2 B/element ≈ 196 KB), which is negligible compared to the ≈95 ms decode step time. Expert routing, which during prefill spans the full prompt, also runs on-device during decode.

The Mamba-2 recurrent step (Equations 1–2) runs entirely on-device. The SSM state **h** lives in a fixed DRAM location and is updated in-place at every decode step, so it is never transferred to the host.

3.2 Mesh Sharding and Fabric

The tiny model uses a 1×4 mesh (one row of 4 chips). As noted in Section 2.1, hardware trace requires all inter-chip communication to run along a single row or column – a 1×4 mesh satisfies this because all 4 chips sit in one row. Four chips also provide enough DRAM to hold the tiny model's weights (Section 2.5). With fabric enabled, different parts of the model are split across the 4 chips in different ways:

- **MoE experts**: split evenly across the 4 chips (expert parallelism from Section 2.4). After each chip computes its local partial result, the partial vectors are collected over the inter-chip links and summed.
- **Mamba projections**: W_{in} has shape $H \times M$: the input is an H-element hidden state vector $\mathbf{x}$, and multiplying it by W_{in} produces an M-element output vector $\mathbf{y}$, where each output value $y_j = \mathbf{x} \cdot W_{:,j}$ (dot product of the full input with column j).
  To illustrate with a small example, suppose H=4, M=4, N=2 chips, input $\mathbf{x} = [x_1, x_2, x_3, x_4]$, and W_{in} with entries W_{ij}. The correct output is $y_j = x_1W_{1j} + x_2W_{2j} + x_3W_{3j} + x_4W_{4j}$.
  Vertical split (by columns, what this implementation uses): chip 0 holds columns 1–2, chip 1 holds columns 3–4. Both chips have the full input $\mathbf{x}$ and each computes complete dot products for its own columns:

  $$\text{chip 0}: y_1 = \mathbf{x} \cdot W_{:,1}, \quad y_2 = \mathbf{x} \cdot W_{:,2}$$
  $$\text{chip 1}: y_3 = \mathbf{x} \cdot W_{:,3}, \quad y_4 = \mathbf{x} \cdot W_{:,4}$$

  Each chip produces a different part of the output with no overlap, so the results are **concatenated**: $[y_1, y_2, y_3, y_4]$.
  Horizontal split (by rows): chip 0 holds rows 1–2, chip 1 holds rows 3–4. Neither chip has all the rows needed to complete a dot product, so each produces a *partial* sum for every output value:

  $$\text{chip 0}: \tilde{y}_j = x_1W_{1j} + x_2W_{2j} \quad \text{for } j = 1, 2, 3, 4$$
  $$\text{chip 1}: \hat{y}_j = x_3W_{3j} + x_4W_{4j} \quad \text{for } j = 1, 2, 3, 4$$

  Both chips contribute a piece of every output value, so the results must be **summed element-wise** to recover $y_j = \tilde{y}_j + \hat{y}_j$ for all j.
  Concatenation is cheaper than an element-wise sum across chips, so the vertical split is preferred. The same column split is applied to W_{out}.
- **All-gather after each projection.** After each sharded projection, the inter-chip links perform an *all-gather*: each chip sends its local piece to every other chip, so all chips end up with the complete result. For the Mamba vertical split, this means chip 0 sends $[y_1, y_2]$ to chip 1 and receives $[y_3, y_4]$ back – both chips end up with $[y_1, y_2, y_3, y_4]$. For MoE, the all-gather instead accumulates the partial sums so every chip holds $y_j = \tilde{y}_j + \hat{y}_j$.
- **Attention, shared MLP, norms – not sharded.** Recall from Section 2.3 that the Mamba block produces its output $\mathbf{y}$ (Equation 2), which is projected back to hidden size H by W_{out} and added to the original hidden state $\mathbf{x}$ as a residual connection – giving a new H-element hidden state. After the all-gather on W_{out}'s output, every chip holds this same

complete H-element vector and can run these components independently with no further communication. Every chip therefore stores an identical full copy of these weights:

  - *Attention* (4 layers): Q and O project $H \to H$ (1536×1536); K and V use grouped-query attention and project $H \to 512$ (1536×512 each).
    Total: (2×1536×1536 + 2×1536×512)×2 B ≈ 12.6 MB per layer.
  - *Shared MLP* (40 layers): gate+up (1536×1024) and down (512×1536). Total: (1536×1024+512×1536)×2 B ≈ 4.7 MB per layer.
  - *RMS norms*: RMS norm normalises the hidden state, then scales each of the H dimensions by a learned weight: H×2 B = 1536×2 = 3 KB per norm.

  All affordable within the 12 GB per-chip budget.
- **LM head**: the final projection from hidden size H to vocabulary size (100,352 words) is column-sharded across the 4 chips, so each chip holds a H×25,088 slice and computes logits for its share of the vocabulary. The partial logits slices are concatenated over the inter-chip links to reconstruct the full logit vector before token selection.

3.3 Decode Trace: Warmup, Capture, and Replay

As described in Section 2.1.4, a *trace* records the full sequence of operations once (*capture*), then replays it with a single device command (*replay*), eliminating the per-operation PCIe overhead. Before capture, however, the model must run at least two non-trace decode steps (*warmup*):

- **Step 1 – kernel compilation.** Each operation (e.g. a matrix multiply) is implemented as a low-level program called a *kernel*. TTNN compiles each kernel for a specific tensor shape – the compiled code has the input dimensions hard-coded for performance, so a kernel compiled for a 256×H input tensor (prefill, 256 tokens) cannot be reused for a 1×H input tensor (decode, 1 token). TTNN compiles lazily: it only compiles a kernel the first time that shape is encountered. In the benchmark, a short warm-up prefill runs first to compile all prefill-shape kernels. The decode warmup steps then follow; the first one triggers compilation of all decode-shape kernels.
- **Step 2 – address stabilisation.** The trace records the DRAM address of the SSM state tensor **h** so it can read and update it during every replay. TTNN may assign **h** its final address only after the first decode step completes. The second warmup step ensures this address has settled; if the trace were captured after only one step, it might record the wrong address and read garbage values during every replay.

After warmup, capture begins. A trace records operations with their DRAM addresses fixed at capture time. Each token is represented as a vector of size H (the embedding table maps a token ID to an H-element vector, which feeds directly into the first decoder layer as the hidden state). Because all token embeddings have this same size H, the host can reuse the same fixed-address DRAM buffer (`_decode_trace_input`) for every replay – it simply overwrites the buffer with the new token's embedding values before each replay. A trace is a list of operations to re-execute. At replay time, each operation re-reads from the same DRAM addresses that were recorded during capture. The first operation in the trace is:

$$\mathbf{h} \leftarrow \texttt{ttnn.add}(\texttt{\_decode\_trace\_input}, \texttt{\_trace\_zeros}) \quad (3)$$

where `_trace_zeros` is a second fixed-address buffer filled with zeros. This add is a no-op (adding zeros changes nothing), but it is the operation that makes the trace re-read from `_decode_trace_input` at replay time. The host writes the new token's embedding into that buffer just before each replay, and this add picks it up.

The benchmark script follows this sequence:

(1) Run prefill on the full prompt (split into chunks of 256 if longer).
(2) Run 2 non-trace decode steps (warmup).
(3) Capture the trace with a zeros embedding.
(4) Each decode step: overwrite `_decode_trace_input` with the token embedding, then replay the trace with a single device command.

3.4 CPU and CUDA Baselines with `torch.compile`

To validate correctness and provide a performance comparison against the TT hardware implementation, I run the same Granite models through the standard HuggingFace pipeline on CPU and CUDA. This baseline optionally applies `torch.compile` to close the gap between interpreted PyTorch and an optimised backend.

CUDA: Three compiler options are configured:

- **Triton autotuning** (`max_autotune_gemm`): Triton is the GPU kernel compiler used by `torch.compile`. A GPU executes threads in groups of 32 called *warps*; multiple warps are organised into a *thread block*, which is the unit of work assigned to one set of GPU cores. For a matrix multiply, a thread block handles a sub-matrix called a *tile*: it loads that tile from global memory into fast on-chip shared memory, computes the partial products, then moves to the next tile. There are many valid tile size choices (e.g. 64×64, 128×64, 256×32), and the fastest choice depends on the matrix dimensions and the specific GPU. With autotuning enabled, Triton benchmarks a set of candidate tile sizes and keeps the fastest one; this search runs once during the warmup pass and the winner is reused for all subsequent calls with the same shape.
- **Shape padding** (`shape_padding`): as established earlier, a kernel is a low-level program compiled for a specific operation. The GPU runs one thread per element, each thread with its own index, and groups 32 threads into a warp that executes together. If the matrix has 33 columns, two warps are launched: warp 1 contains 32 threads for indices 0–31, warp 2 contains 32 threads for indices 32–63. But only index 32 is a real column; indices 33–63 do not exist. Each thread computes a memory address from its index and accesses it – the kernel has no automatic awareness of the matrix boundary. The 31 threads with indices 33–63 will therefore access memory beyond the matrix, overwriting

whatever data is stored there. To prevent this, the kernel must include an explicit guard: `if (col < N) { access memory }`. This guard is part of the kernel, so every thread in every warp executes it on every memory access, including all the valid threads in warp 1.
Padding appends zeros to round the dimension up to the next multiple of 32 (e.g. H=1536 → 2048). Every group is then fully covered by valid elements, no copy ever exceeds the boundary, and the compiler removes the guard entirely.

- **CUDA graph replay disabled**: a CUDA graph, like TTNN's hardware trace, captures a sequence of GPU operations and replays them with a single launch, but requires all memory addresses to be fixed at capture time. The Mamba SSM state and the attention KV cache are updated in-place every step and their addresses change with sequence position, so they cannot be frozen into a static graph.

CPU: With `cpp_wrapper` enabled, `torch.compile` generates a C++ entry point that calls the compiled kernels directly. Without it, the compiled code is still reached through the Python interpreter, which adds a per-call dispatch cost on every operation along the decode path. The C++ wrapper eliminates this overhead.

On CUDA, after compilation a *warmup* pass runs two representative prompt lengths through a full prefill and 20 decode steps each, so all Triton kernels are compiled and cached before timing begins.

4 Porting Challenges

Porting from the HuggingFace reference implementation to on-device TTNN execution required resolving challenges in three categories.

4.1 Numerical Precision

4.1.1 Tile-layout quantisation. TTNN operates on 32-element tiles and requires all tensors in bfloat16 (or lower) tile layout. HuggingFace uses 32-bit float accumulation in several places (notably SSM state updates and normalisation). Converting these to bfloat16 without extra precision guards caused top-1 token predictions to diverge from the HuggingFace reference within the first few decode steps across all five evaluation prompts. The solution was to enable higher-precision accumulation on RMSNorm and matrix-multiply operations (`fp32_dest_acc_en=True`), and keep the SSM state in bfloat16 with high-precision internal accumulators. After applying these guards, the tiny model produces coherent, on-topic responses that match the A100 reference for all five prompts (Section 7.2.5).

4.1.2 MoE weight quantisation. For the small model, MoE expert weights are stored in `bfloat8_b` (8-bit block float) to reduce per-device DRAM from ∼36 MB/layer to ∼18 MB/layer, a necessary compression given the 12 GB DRAM per chip. The tiny model's MoE weights remain in bfloat16, as its smaller hidden size (1536 vs. 4096) means the memory saving does not justify the quantisation overhead. Activations remain in bfloat16 for both models.

The small model produces coherent, on-topic responses across all five evaluation prompts under this quantisation (Section 7.2.5).

4.1.3 Logits gather axis on multi-chip mesh. As described in Section 3.2, the LM head weight is column-sharded across the 4 chips, so each chip produces logits for only its slice of the vocabulary. TTNN's `ConcatMeshToTensor` composer requires explicitly specifying the concatenation axis to reconstruct the full logit vector. Using the wrong axis (dim=0 instead of dim=3 in TTNN's 4D tile layout) produced scrambled logits that generated coherent-looking but semantically wrong tokens. This was diagnosed by comparing token-id sequences between the TT and HuggingFace paths and corrected for both models.

4.2 Architectural Adaptation

4.2.1 Mamba2 state persistence. HuggingFace's `MambaCache` stores the SSM state as a list of tensors indexed by layer, updated via `.copy_()` into a pre-allocated PyTorch tensor. TTNN requires a persistent fixed-address DRAM tensor because trace replay records specific buffer addresses. The port replaced the list-based cache with a `ttnn.Tensor` per layer, and the state update writes the new state into the same fixed-address buffer via `ttnn.assign` rather than reallocating a new tensor, preserving the address recorded in the trace so every replay reads and writes the correct location.

A second problem came from how TTNN manages tensor memory. TTNN tracks how many references point to each tensor; freeing a tensor that is still referenced elsewhere causes the device to silently read from freed memory and return zeros. The original code freed the old state tensor and created a new one, which also assigned a new DRAM address to `self._ssm_state_tt`. The trace, which had recorded the old address, then read from the wrong location on every replay. Writing the updated state into the original buffer via `ttnn.assign` fixes both issues: the old address is kept, and no tensor is freed while still in use.

4.2.2 MoE expert sharding mismatch. Granite-4.0-h uses 64 experts for the tiny model and 72 experts for the small model. On a 1×4 mesh, 64 divides evenly into 16 experts per device. On an 8×1 mesh, 72 divides into 9 experts per device. Each device runs all its local experts and multiplies each output by the corresponding routing weight; unselected experts have a routing weight of zero and therefore contribute nothing to the sum. This differs from the HuggingFace implementation, which skips unselected experts entirely, and required re-implementing the MoE forward pass from scratch using TTNN matmuls and an explicit routing-weight multiply.

4.3 Toolchain Limitations

4.3.1 No native Mamba2 chunk-scan kernel. The HuggingFace Mamba2 implementation uses two Triton kernels for decode: `causal_conv1d_update` (a fused causal convolution step) and `selective_state_update` (a fused recurrent state update). Neither has an equivalent in the TTNN Metal kernel library at the time of writing. Both were initially implemented using sequential TTNN ops, and two custom Metal kernels — `conv1d_decode` and `ssm_update` — were written to replace them. Both are described in Section 5.

4.3.2 Trace capture must run outside the generation loop. As described in Section 2.1.4, during trace capture operations are stored but not executed on the device. This means the model state — the SSM recurrent state and the attention past-token cache — is not updated during capture. If capture is triggered inside the generation loop at step t, the device state is still from step $t-1$ when the trace

runs, so all replays produce the wrong output: every generated token is shifted by one position. This was confirmed by observing that the output with trace at step t matched the output without trace at step $t-1$.

The fix is to capture the trace once before generation begins, using a dummy (zeros) input that does not affect the model state. The capture procedure is described in Section 3.3.

5 Optimisation

5.1 Fused SSM State-Update Kernel

Because each TTNN operation is a separate command to the device, sequential operations on the same tensor each read it from DRAM independently — there is no automatic reuse of the value from the previous operation. Without the custom kernel, the Mamba-2 recurrent step (Equations 1–2) is implemented as four sequential TTNN operations, each reading the full $H_m \times D \times N$ state tensor from DRAM: (1) compute $\exp(\Delta \odot A) \odot \mathbf{h} + (\Delta\, \sigma) \odot B$ to get the new state, (2) write the new state back to the fixed-address DRAM buffer, (3) multiply the state element-wise by C (which selects what to extract from memory), and (4) sum over N to produce the per-head output.

Before the kernel is called, the projection step (Section 2.3) has already combined Δ, σ, B, and A into two tensors: $dBx = (\Delta\, \sigma) \odot B$ (the new token contribution from Equation 1) and $dA = \exp(\Delta \odot A)$ (the forget factor). Both have shape $H_m \times D \times N$ and are what the kernel receives.

The custom Metal kernel `ssm_update` fuses all four steps into a single DRAM pass. Algorithm 1 shows its structure. Rather than writing intermediate results back to DRAM, the kernel keeps them in on-chip L1 circular buffers, writing only the updated state $\mathbf{h}$ and the output $\mathbf{y}$ to DRAM. This eliminates three redundant DRAM reads and replaces four separate host-to-device commands with one per Mamba layer.

Algorithm 1 SSM update kernel (`ssm_update`)

Require: $dBx = (\Delta\, \sigma) \odot B$, $dA = \exp(\Delta \odot A) \in \mathbb{R}^{H_m \times D \times N}$ (from Eq. 1, in DRAM)
Require: $\mathbf{h} \in \mathbb{R}^{H_m \times D \times N}$ (current SSM state, in DRAM)
Require: $C \in \mathbb{R}^{H_m \times N}$ (selects what to extract from state, Eq. 2, in DRAM)
Ensure: $\mathbf{h}$ updated in-place (written back to DRAM)
Ensure: Output $\mathbf{y} \in \mathbb{R}^{H_m \times D}$ (written to DRAM)
▷ All intermediate values stay in L1 circular buffers
for all N-tiles n **do** ▷ **Phase 1**: update state
  Read $dBx[:, n]$, $dA[:, n]$, $\mathbf{h}[:, n]$ from DRAM → L1
  $\mathbf{h}[:, n] \leftarrow dA[:, n] \odot \mathbf{h}[:, n] + dBx[:, n]$ (in L1)
  Write $\mathbf{h}[:, n]$ to DRAM; keep copy in L1 staging buffer
end for
for all N-tiles n **do** ▷ **Phase 2**: extract output using C
  $\mathbf{yc}[:, n] \leftarrow \mathbf{h}[:, n] \odot C[:, n]$ (row broadcast over D, in L1)
end for
$\mathbf{y}[:] \leftarrow \sum_n \mathbf{yc}[:, n]$ ▷ **Phase 3**: reduce over N, write $\mathbf{y}$ to DRAM

5.2 Fused Convolution Kernel

Recall from Section 2.3 that the convolution cache is a K=4 row tensor, where each row holds one token's σ, B, and C values concatenated. Row 0 is the oldest token; row 3 is the most recent. Each decode step must: (1) remove the oldest row and move the remaining three rows down, (2) write the current token's σ, B, C values into the now-empty row 3, (3) compute the weighted sum $\sum_{k=0}^{K-1} row[k] \odot w[k]$ (element-wise, summed over the K lags), (4) add a learned bias, and (5) apply SiLU activation.

Without the custom kernel, the shift requires three separate copy operations (row 1 → row 0, row 2 → row 1, row 3 → row 2), followed by one write to insert the new token into row 3. The weighted sum requires four element-wise multiplies ($row[0] \odot w[0]$, $row[1] \odot w[1]$, $row[2] \odot w[2]$, $row[3] \odot w[3]$) and three adds to sum them (summing four values always takes three additions). Bias and SiLU each add one more operation — twelve host-to-device commands in total, each reading the cache tensor from DRAM.

The custom Metal kernel `conv1d_decode` fuses all five steps into a single host-to-device command. It reads the old cache and the new input once from DRAM, performs the shift, weighted sum, bias add, and SiLU entirely in L1, and writes the updated cache and output back to DRAM in one pass. Like `ssm_update`, it uses pre-allocated fixed-address output buffers so the result is trace-safe.

5.3 Whole-Model Decode Trace

Without trace, every TTNN operation in a single decode step requires a separate host-to-device round-trip over PCIe, and the cumulative overhead dominates step latency. The Metal trace mechanism (described in Section 3.3) records the entire forward pass once and replays it with a single device command. Algorithm 2 summarises the warmup, capture, and replay sequence.

Algorithm 2 Decode trace: warmup, capture, and replay

Initialisation (once, before generation)
Run one prefill pass to compile all prefill-shape kernels
Run 2 non-trace decode steps ▷ step 1: compile decode-shape kernels; step 2: stabilise DRAM addresses
Allocate fixed-address buffer `_decode_trace_input` (size H, all zeros)
Begin trace capture
  $\mathbf{h} \leftarrow$ `_decode_trace_input` + $\mathbf{0}$
  Run full forward pass
End trace capture

Generation loop (one step per token)
for all decode steps $t = 1, 2, \ldots$ **do**
  Write token t's embedding into `_decode_trace_input` via `ttnn.assign`
  `ttnn.execute_trace()`
  Read logits; select next token on host CPU
end for

5.4 Chunked-Prefill Chunk Size Tuning

Prefill is processed in chunks of C tokens. To choose the best value, $C \in \{192, 256, 320, 384, 512\}$ was tested on both models running

on TT hardware with both fused kernels and trace enabled. The five evaluation prompts produce 5, 8, 25, 96, and 176 tokens after Granite tokenisation. Table 3 shows TTFT for the two longest (96 and 176 tokens, where chunk size could matter) and mean decode throughput across all five.

Table 3: Effect of prefill chunk size on TTFT and decode throughput (TT hardware, both fused kernels and trace enabled).

Model	Chunk	TTFT (ms)		Decode
	size	96-token prompt	176-token prompt	(tok/s)
tiny	192	1112.9	1675.9	10.64
	256	1104.7	1663.8	**10.67**
	320	1114.6	1669.2	10.65
	384	**1104.6**	**1656.1**	10.63
	512	1109.7	1666.6	10.65
small	192	1911.1	2808.9	**5.86**
	256	1917.6	**2789.6**	**5.86**
	320	1909.6	2825.1	5.85
	384	**1901.6**	2836.8	5.84
	512	1939.0	2823.5	**5.86**

All values produce essentially identical TTFT and decode throughput for both models, so chunk size has no measurable effect. C=256 is used as the default: it achieves the highest mean decode throughput for the tiny model (10.67 tok/s) and ties for the highest for the small model (5.86 tok/s).

6 Related Work

Previous work has explored LLM deployment on GPUs, TPUs, and custom accelerators. Systems such as TensorRT and XLA aim to optimize execution by compiling models to hardware-specific representations. Mamba-series models [Dao and Gu 2024; Gu and Dao 2023] have been shown to match transformer quality at reduced inference cost on GPUs, but their recurrent decode is harder to run efficiently on Tenstorrent hardware. Each decode step must update a hidden state, read from it, and produce an output — three dependent operations that cannot be merged without a custom kernel. In TTNN, each operation is a separate command sent from the host CPU to the device, and at batch size 1 the data processed is small enough that sending the command takes longer than the computation itself. A transformer step spends most of its time in large matrix multiplications (attention and feed-forward), so the per-command cost is negligible compared to the actual computation. Expert-parallel inference for sparse MoE models has been studied for GPU clusters [Lepikhin et al. 2021], but the interaction between collective operations and trace replay has not been characterised for Wormhole-class hardware.

7 Evaluation

7.1 Experimental Setup

7.1.1 Hardware. The CPU and Wormhole devices share the same host machine (AMD EPYC 9354P, 32 cores at 3.8 GHz with 2-way hyperthreading, 566 GB DDR5 RAM). The A100 is located in a separate server (NVIDIA A100 PCIe 80 GB [NVIDIA 2020], 1,935 GB/s HBM2e bandwidth, 312 TFLOPS BF16; host: Intel Xeon Icelake, 14 cores, 117 GB RAM) on the same network.

7.1.2 Software. All Python code runs under Python 3.10.12. The Wormhole backend uses TTNN 0.67.0 (tt-metalium). The HuggingFace baselines use `transformers` 4.57.1 with PyTorch 2.3. The A100 node runs NVIDIA driver 595.45.04 with CUDA 13.2.

7.1.3 Configurations. Four primary configurations are used as baselines:

- **TT-tiny**: Granite-4.0-h-tiny on 1×4 Wormhole submesh (4 chips, fabric enabled). All weights `bfloat16`. Decode trace enabled.
- **TT-small**: Granite-4.0-h-small on 8×1 Wormhole submesh (8 chips, fabric enabled). MoE weights in `bfloat8_b`, all others `bfloat16`. Decode trace enabled.
- **CUDA-tiny / CUDA-small**: HuggingFace reference on the NVIDIA A100 80 GB GPU, `bfloat16`, without `torch.compile`. A compiled variant (`torch.compile` with the `inductor` backend) was also run for comparison (see Section 7.3).
- **CPU-tiny / CPU-small**: Same HuggingFace reference on the host machine CPU, `float32`, without `torch.compile`. A compiled variant was also run for comparison.

In total, 24 benchmark runs were performed: 10 on Tenstorrent with both kernels enabled (2 models × 5 chunk sizes: 192, 256, 320, 384, 512), 6 kernel ablation runs (2 models × 3 kernel configurations: no kernels, conv1d only, SSM only), 4 on CUDA (2 models × with/without `torch.compile`), and 4 on CPU (same). The default chunk size of 256 and the uncompiled CUDA and CPU paths are used as the primary baselines throughout the paper.

Five prompts of increasing length are used: 5, 8, 25, 96, and 176 tokens. Each backend generates 20 tokens per prompt. Let t_i be the time (in seconds) from submitting the i-th input token to receiving the corresponding logits (i.e., the time to produce one output token). Decode throughput is

$$\text{tok/s} = \frac{1}{\frac{1}{19}\sum_{i=2}^{20} t_i}, \tag{4}$$

where step 1 is excluded because it is slightly slower after a cache reset. At each step, a repetition penalty p=1.3 is applied to the logit z_j of every token j that has already appeared in the output:

$$z_j \leftarrow \begin{cases} z_j / p & \text{if } z_j > 0 \\ z_j \times p & \text{if } z_j \le 0. \end{cases} \tag{5}$$

Dividing a positive logit by p>1 lowers it, and multiplying a negative logit by p makes it more negative. Both operations reduce the probability of repeating already-seen tokens.

7.2 Results

7.2.1 Kernel ablation. Both fused kernels contribute independently to decode throughput. For the tiny model, `conv1d_decode` adds ≈0.08 tok/s and `ssm_update` adds ≈0.04 tok/s over the no-kernel baseline; combined they add ≈0.15 tok/s (≈1.4%). For the small model, `conv1d_decode` adds ≈0.07 tok/s. `ssm_update` alone provides no measurable gain but contributes an additional ≈0.04 tok/s

Table 4: Decode throughput (tok/s) by fused kernel configuration, averaged across all five prompts (C=256). Both kernels active is the default used in all subsequent tables.

Fused kernel configuration	tiny (tok/s)	small (tok/s)
No fused kernels	10.52	5.75
`conv1d_decode` only	10.60	5.82
`ssm_update` only	10.56	5.75
Both kernels (default)	**10.67**	**5.86**

when combined with `conv1d_decode`. The absolute gains are modest because generating one token at a time means most of the time is spent reading model weights from memory, not doing computation — so reducing the number of operations per Mamba2 step only saves a small fraction of the total step time.

Table 5: Decode throughput (tok/s), tiny model. Higher is better. *Token counts for TT/CPU tokenizer; A100 tokenizer produces 98 and 181 tokens for the same prompts.

Backend	short_8 (5 tok)	short_10 (8 tok)	med_32 (25 tok)	long_128 (96 tok*)	long_256 (176 tok*)
CPU	2.54	2.56	2.55	2.55	2.52
CPU + `torch.compile`	2.45	2.45	2.45	2.46	2.45
A100	8.99	9.04	8.49	8.58	8.94
A100 + `torch.compile`	6.15	5.96	5.60	5.27	5.75
WH 1×4 (trace, C=256)	**10.68**	**10.69**	**10.65**	**10.67**	**10.66**

Table 6: Prefill TTFT (ms), tiny model. Lower is better. *See Table 5 footnote.

Backend	short_8 (5 tok)	short_10 (8 tok)	med_32 (25 tok)	long_128 (96 tok*)	long_256 (176 tok*)
CPU	11232	11330	11479	12065	12770
CPU + `torch.compile`	11942	12095	12148	12175	12334
A100	**321**	**322**	**370**	**364**	**397**
A100 + `torch.compile`	391	331	389	444	450
WH 1×4 (trace, C=256)	730	735	889	1105	1664

7.2.2 Granite-4.0-h-tiny. The Wormhole tiny model (10.65–10.69 tok/s) exceeds the A100 (8.5–9.0 tok/s) by ≈18% and CPU (2.52–2.56 tok/s) by ≈4× across all prompt lengths. Decode throughput is constant across prompt lengths because trace replay cost is independent of prefill context length. The A100 without compile shows a dip at medium prompt lengths (med_32, long_128). `torch.compile` on the A100 degrades decode to 5.27–6.15 tok/s (see Section 7.3). Wormhole prefill TTFT (730–1664 ms) is 2–4× slower than the A100 (321–397 ms).

7.2.3 Granite-4.0-h-small. The small model on 8 chips achieves 5.84–5.89 tok/s, below the A100 (7.12–8.15 tok/s) by ≈20% but ≈9× faster than CPU (0.65–0.66 tok/s). As with tiny, Wormhole decode throughput is constant across all prompt lengths. `torch.compile` on the A100 substantially degrades decode (4.83–6.83 tok/s vs. 7.1–8.2 tok/s uncompiled). See Section 7.3. Wormhole prefill TTFT (1408–2790 ms) is 2–4× slower than A100 (737–790 ms).

Table 7: Decode throughput (tok/s), small model. Higher is better. *See Table 5 footnote.

Backend	short_8 (5 tok)	short_10 (8 tok)	med_32 (25 tok)	long_128 (96 tok*)	long_256 (176 tok*)
CPU	0.66	0.65	0.65	0.65	0.65
CPU + `torch.compile`	0.64	0.65	0.65	0.65	0.64
A100	**7.12**	**8.15**	**8.13**	**8.12**	**8.06**
A100 + `torch.compile`	4.83	6.69	6.83	6.81	6.77
WH 8×1 (trace, C=256)	5.89	5.84	5.85	5.85	5.86

Table 8: Prefill TTFT (ms), small model. Lower is better. *See Table 5 footnote.

Backend	short_8 (5 tok)	short_10 (8 tok)	med_32 (25 tok)	long_128 (96 tok*)	long_256 (176 tok*)
CPU	30119	30500	31049	32033	32969
CPU + `torch.compile`	30486	30860	31687	32270	33600
A100	**790**	**737**	**758**	**778**	**789**
A100 + `torch.compile`	839	752	771	791	802
WH 8×1 (trace, C=256)	1408	1491	1838	1918	2790

Table 9: Model load time (seconds). Lower is better. Compile variants (`torch.compile`) are not shown separately: `torch.compile` wraps the model after loading and its JIT compilation is deferred to the first forward pass, so load time is unchanged.

Model	Backend	Load time (s)
tiny	CPU	**1.9**
tiny	A100	37.7
tiny	WH 1×4	23.0
small	CPU	**6.5**
small	A100	14.7
small	WH 8×1	464

7.2.4 Model load time. Both backends load weights from the local HuggingFace cache. Wormhole tiny (23 s) loads faster than the A100 (38 s). Wormhole small (464 s) is substantially slower than the A100 (15 s), reflecting the cost of distributing weights across 8 devices over PCIe.

7.2.5 Output quality. Both models on Wormhole with trace enabled produce coherent, on-topic responses across all five prompts. All five TTNN chunk-size variants produce token-for-token identical outputs, confirming that chunk size does not affect generation quality. Six distinct agreement patterns are observed:

- **WH, CPU, CPU+compile, A100, A100+compile all identical.** Example — tiny, *"The largest planet in our solar system is"*: all backends: *"Jupiter. It has a diameter of about 88,846 miles (142,984 kilometers), which makes…"*
- **WH close to CPU and A100 but differs at the tail.** Agreement through most tokens, divergence on the last few words. Example — tiny, *"The capital of France is"*: WH: *"…skyline and one that many people…"*; CPU / A100: *"…skyline and one among many tourist…"*

- **CPU and CPU+compile always identical.** `torch.compile` does not change output on CPU. Example — tiny, long_128: CPU and CPU+compile both: *"…or other techniques for improved efficiency. However, despite their impressive capabilities, LLMs still face…"*
- **A100 and A100+compile can differ.** Triton autotuning changes floating-point operation order. Example — tiny, long_128: A100: *"…like LSTMs or GATs for improved performance. The training process involves…"*; A100+compile: *"…like LSTMs or GATs (Graph Attention Networks) for improved performance. However, despite…"*
- **CPU and A100 differ on longer prompts.** float32 (CPU) vs. bfloat16 (A100) accumulation diverges over many tokens. Example — tiny, long_128: CPU: *"…or other techniques for improved efficiency…"*; A100: *"…like LSTMs or GATs for improved performance…"*
- **WH differs from both CPU and A100 on longer prompts.** WH uses bfloat16 on different hardware with a different tokenizer. Example — tiny, long_128: WH: *"…like LSTMs or RNNs for improved performance…"*; CPU: *"…or other techniques for improved efficiency…"*; A100: *"…like LSTMs or GATs for improved performance…"*

7.3 Discussion

Tiny model: Wormhole exceeds A100. The tiny model on 4 Wormhole chips (10.65–10.69 tok/s) is ≈18% faster than the A100 (8.5–9.0 tok/s). Decode trace recording (Section 4) replaces the per-token command overhead with a single replay call, which is the key reason Wormhole wins at this scale. The A100 has years of software optimisation behind it, so outperforming it on a relatively new and less mature hardware platform is a strong result.

Small model: Wormhole below A100. The small model on 8 chips (5.84–5.89 tok/s) is ≈20% slower than the A100 (7.1–8.2 tok/s). The bottleneck is reading weights from memory: at this model size, each token generation must load a large amount of data across 8 chips, and the 8×1 mesh has less total memory bandwidth than the A100's HBM2e. Trace is enabled but the memory cost outweighs the command overhead savings.

CPU baselines. Both CPU results (2.5–2.6 tok/s tiny, 0.65–0.66 tok/s small) are far below Wormhole and A100. Wormhole is ≈4× faster than CPU for tiny and ≈9× for small. `torch.compile` gives no meaningful improvement on CPU for this model family (Section 7.3).

Constant decode rate on Wormhole. Wormhole throughput does not change across prompt lengths (10.65–10.69 tok/s for tiny, 5.84–5.89 tok/s for small) because the trace replays the same fixed sequence of operations regardless of how long the input was. The A100 without `torch.compile` dips slightly at medium prompt lengths (med_32, long_128), likely because its GPU kernels are less efficient at those particular input sizes.

Fused kernel gains at larger batch sizes. All benchmarks in this paper use batch size 1, where each token generation step reads the full model weights from memory but does very little arithmetic. When only one token is generated at a time, the time saved by fusing operations is small compared to the total weight-reading cost, which is why the kernel gains are only ≈1–2%. At larger batch sizes, the same weights are shared across multiple requests in a single step, so reading weights becomes less of a bottleneck and the arithmetic savings from fused kernels matter more. The current implementation does not support batched decode, but extending the fused kernels to batch size > 1 is a natural next step and would likely yield more significant gains.

`torch.compile` on CPU and CUDA. `torch.compile` does not improve throughput for either backend. On CPU, compiled decode (2.45–2.46 tok/s tiny, 0.64–0.65 tok/s small) is marginally slower than uncompiled, with similar prefill time after warmup. On CUDA, compiled decode (5.27–6.15 tok/s tiny, 4.83–6.83 tok/s small) is substantially slower than uncompiled (8.5–9.0 tok/s tiny, 7.1–8.2 tok/s small). The main reason is that the Mamba2 recurrent state prevents CUDA graph capture, which is the primary mechanism by which `torch.compile` speeds up GPU inference; without it, the compilation overhead outweighs any gains. The notably lower result for small at short_8 (4.83 tok/s) is because `torch.compile` runs additional setup work the first time it sees a new input size, which slows down that prompt's benchmark. The A100 rows in Tables 5 and 7 therefore use the uncompiled path.

8 Threats to Validity

Several factors may affect how the results should be interpreted:

- All benchmarks use batch size 1 and a fixed sequence of five prompts. Results may not generalise to higher batch sizes, longer sequences, or different prompt distributions.
- The A100 and Wormhole benchmarks were run on separate machines at different times, so differences in background system activity between the two machines may slightly affect the measured numbers.

This work focuses on engineering and performance measurement. No personal or sensitive data is used. Power consumption is not measured in this project.

9 Conclusion

This project ports Granite-4.0-h-tiny and Granite-4.0-h-small to Tenstorrent Wormhole hardware and benchmarks them against an NVIDIA A100 and CPU. The tiny model (4 chips) achieves 10.65–10.69 tok/s, which is ≈18% faster than the A100 (8.5–9.0 tok/s) and ≈4× faster than CPU. The small model (8 chips) achieves 5.84–5.89 tok/s, which is ≈20% below the A100 (7.1–8.2 tok/s) because reading the larger model's weights across 8 chips takes more time than the A100 can match at this scale, but is still ≈9× faster than CPU.

The key enabler for the tiny model result is decode trace: recording the full set of operations once and replaying it for every token removes the per-token overhead of sending commands from the host to the device. A critical implementation detail is that trace recording must happen outside the generation loop — recording inside the loop causes all state updates to lag by one step, producing incorrect output. This was fixed by capturing the trace in a separate setup step before generation begins.

Two custom fused kernels (`ssm_update` and `conv1d_decode`) reduce the number of memory reads per Mamba2 step. At batch size 1 the gains are modest (≈1–2%) because most time is spent reading weights from memory rather than doing arithmetic, but the

gains are expected to be larger at higher batch sizes. Prefill chunk size tuning shows that C=256 gives the best decode throughput.

The main challenges encountered during porting — including weight quantisation, SSM state management across chunked prefill, expert-parallel weight sharding, and collective operation ordering — are documented in Section 4 as a reference for future ports of similar hybrid models to Wormhole hardware.

References

Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. arXiv:2405.21060

Albert Gu and Tri Dao. 2023. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752

IBM. 2025. IBM Granite 4.0: Hyper-Efficient, High-Performance Hybrid Models. https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models. Accessed: 2025.

IBM Research. 2025a. Granite-4.0-h-small Model Card. https://huggingface.co/ibm-granite/granite-4.0-h-small. Accessed: 2025.

IBM Research. 2025b. Granite-4.0-h-small `config.json`. https://huggingface.co/ibm-granite/granite-4.0-h-small/blob/main/config.json. Accessed: 2025.

IBM Research. 2025c. Granite-4.0-h-small `model.safetensors.index.json`. https://huggingface.co/ibm-granite/granite-4.0-h-small/blob/main/model.safetensors.index.json. Accessed: 2025; `metadata.total_size` field gives total checkpoint bytes in bfloat16.

IBM Research. 2025d. Granite-4.0-h-tiny Model Card. https://huggingface.co/ibm-granite/granite-4.0-h-tiny. Accessed: 2025.

IBM Research. 2025e. Granite-4.0-h-tiny `config.json`. https://huggingface.co/ibm-granite/granite-4.0-h-tiny/blob/main/config.json. Accessed: 2025.

IBM Research. 2025f. Granite-4.0-h-tiny `model.safetensors.index.json`. https://huggingface.co/ibm-granite/granite-4.0-h-tiny/blob/main/model.safetensors.index.json. Accessed: 2025; `metadata.total_size` field gives total checkpoint bytes in bfloat16.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. arXiv:2006.16668

NVIDIA. 2020. NVIDIA A100 Tensor Core GPU Architecture. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf. Accessed: 2025.

Tenstorrent. 2023a. Galaxy – 32-Chip Wormhole System. https://tenstorrent.com/hardware/galaxy. Accessed: 2025.

Tenstorrent. 2023b. Wormhole Architecture Overview. https://tenstorrent.com/hardware/wormhole. Accessed: 2025.